

Лабораторная работа №1

Студент: Мансуров Роман

Группа: 6312-100503D

1. Цель работы

Изучение алгоритма умножения квадратных матриц, исследование зависимости времени выполнения от размера матриц, оценка производительности классического алгоритма умножения с тремя вложенными циклами, а также получение статистических характеристик времени выполнения.

2. Описание работы алгоритма

В данной работе реализован классический алгоритм умножения двух квадратных матриц. Для матриц A и B размера $n \times n$, результирующая матрица C вычисляется по формуле:

$$C_{ij} = \sum_{n=1}^k A_{in} * B_{nj}$$

Алгоритм использует три вложенных цикла:

- Внешний цикл по строкам матрицы A (индекс i)
- Средний цикл по столбцам матрицы B (индекс j)
- Внутренний цикл по элементам строки A и столбца B

Вычислительная сложность алгоритма составляет $O(n^3)$, количество арифметических операций равно $2n^3$ (n^3 умножений и n^3 сложений).

Программа на C++:

1. Читает две матрицы из файлов data/matrix_a.txt и data/matrix_b.txt
2. Выполняет умножение с замером времени с помощью `std::chrono::steady_clock`
3. Сохраняет результат в файл data/result_cpp.txt
4. Выводит размер матрицы и время выполнения

Python-скрипт автоматизирует эксперимент:

1. Генерирует случайные матрицы для размеров 100, 200, 300, 400, 500, 700, 1000

2. Запускает C++ программу 10 раз для каждого размера
3. Верифицирует результат с помощью NumPy
4. Вычисляет среднее время и 95% доверительный интервал
5. Строит график зависимости времени от размера матрицы

3. Характеристики вычислительной системы

Процессор:

- Модель: AMD Ryzen 7 2700 Eight-Core Processor
- Количество ядер: 8
- Тактовая частота: 3.20 GHz

Оперативная память:

- Объем: 24 Гб
- Тип: DDR4

Операционная система:

- Название: Windows 11
- Разрядность: 64bit

4. Результаты экспериментов

Размер 100x100 Запуск 1/10: 0.0010 сек Запуск 2/10: 0.0010 сек Запуск 3/10: 0.0010 сек Запуск 4/10: 0.0005 сек Запуск 5/10: 0.0010 сек Запуск 6/10: 0.0010 сек Запуск 7/10: 0.0010 сек Запуск 8/10: 0.0010 сек Запуск 9/10: 0.0010 сек Запуск 10/10: 0.0010 сек Верификация: успешно	Размер 200x200 Запуск 1/10: 0.0070 сек Запуск 2/10: 0.0061 сек Запуск 3/10: 0.0061 сек Запуск 4/10: 0.0070 сек Запуск 5/10: 0.0061 сек Запуск 6/10: 0.0066 сек Запуск 7/10: 0.0070 сек Запуск 8/10: 0.0073 сек Запуск 9/10: 0.0062 сек Запуск 10/10: 0.0060 сек Верификация: успешно	Размер 300x300 Запуск 1/10: 0.0248 сек Запуск 2/10: 0.0243 сек Запуск 3/10: 0.0243 сек Запуск 4/10: 0.0233 сек Запуск 5/10: 0.0245 сек Запуск 6/10: 0.0233 сек Запуск 7/10: 0.0236 сек Запуск 8/10: 0.0232 сек Запуск 9/10: 0.0243 сек Запуск 10/10: 0.0247 сек Верификация: успешно	Размер 400x400 Запуск 1/10: 0.0608 сек Запуск 2/10: 0.0600 сек Запуск 3/10: 0.0592 сек Запуск 4/10: 0.0610 сек Запуск 5/10: 0.0601 сек Запуск 6/10: 0.0598 сек Запуск 7/10: 0.0598 сек Запуск 8/10: 0.0675 сек Запуск 9/10: 0.0689 сек Запуск 10/10: 0.0593 сек Верификация: успешно
Размер 500x500 Запуск 1/10: 0.1235 сек Запуск 2/10: 0.1230 сек Запуск 3/10: 0.1239 сек Запуск 4/10: 0.1228 сек Запуск 5/10: 0.1243 сек Запуск 6/10: 0.1243 сек Запуск 7/10: 0.1244 сек Запуск 8/10: 0.1239 сек Запуск 9/10: 0.1286 сек Запуск 10/10: 0.1317 сек Верификация: успешно	Размер 700x700 Запуск 1/10: 0.3444 сек Запуск 2/10: 0.3512 сек Запуск 3/10: 0.3375 сек Запуск 4/10: 0.3482 сек Запуск 5/10: 0.3583 сек Запуск 6/10: 0.3634 сек Запуск 7/10: 0.3436 сек Запуск 8/10: 0.3440 сек Запуск 9/10: 0.3472 сек Запуск 10/10: 0.3451 сек Верификация: успешно	Размер 1000x1000 Запуск 1/10: 1.6457 сек Запуск 2/10: 1.4620 сек Запуск 3/10: 1.2386 сек Запуск 4/10: 1.5083 сек Запуск 5/10: 1.3774 сек Запуск 6/10: 1.8298 сек Запуск 7/10: 1.6264 сек Запуск 8/10: 1.3221 сек Запуск 9/10: 1.2317 сек Запуск 10/10: 1.3388 сек Верификация: успешно	

Размер матрицы	Среднее время (сек)	95% доверительный интервал (сек)
100 × 100	0.0010	[0.0008, 0.0011]
200 × 200	0.0065	[0.0062, 0.0069]
300 × 300	0.0240	[0.0236, 0.0245]
400 × 400	0.0616	[0.0591, 0.0642]
500 × 500	0.1250	[0.1230, 0.1271]
700 × 700	0.3483	[0.3429, 0.3537]
1000 × 1000	1.4581	[1.3185, 1.5977]

Время выполнения растёт пропорционально кубу размера, что соответствует теоретической сложности $O(n^3)$.

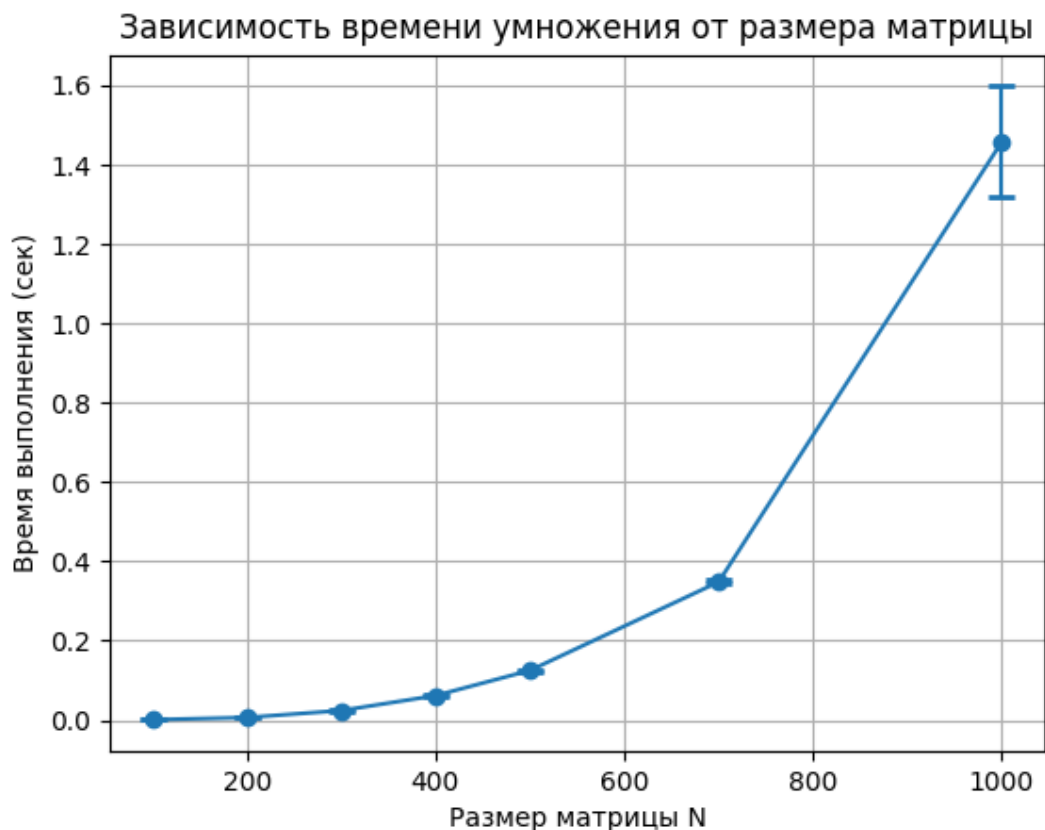
Для размеров до 700×700 наблюдается хорошее соответствие теоретической сложности. Для 1000×1000 наблюдаются отклонения, связанные с влиянием кэш-памяти и операционной системы.

Производительность стабильна ($\sim 2 \times 10^9$ операций/сек) для размеров до 700, снижается до 1.4×10^9 для 1000 из-за ограничений кэш-памяти.

95% доверительные интервалы:

- Для размеров ≤ 700 : узкие ($\pm 2-5\%$), высокая точность
- Для 1000: широкий ($\pm 10\%$), требуется больше измерений для повышения точности

5. График



6. Вывод

В ходе выполнения лабораторной работы была разработана программа для умножения квадратных матриц на языке C++ с использованием классического алгоритма трёх вложенных циклов. Проведено исследование зависимости времени выполнения программы от размера входных данных. Эксперименты показали, что время выполнения растёт пропорционально кубу размера матрицы, что полностью соответствует теоретической оценке сложности $O(n^3)$ алгоритма умножения матриц. Для автоматизации эксперимента, сбора статистических данных и верификации результатов был реализован скрипт на языке Python с использованием библиотек NumPy и SciPy, который выполнил многократные запуски программы, рассчитал среднее время выполнения и доверительные интервалы, а также подтвердил корректность вычислений путём сравнения с эталонной реализацией.\

7. Исходный код

Исходный код представлен в репозитории на GitHub.